

Lab 4 – VibeCoding

Name: Praakruthi PS

SRN: PES1UG24CS324

Section: “5F”

GitHub Copilot Chat History

Task1:

User: I want to work on Task 1 from the README: Consistent coordinate handling.

First inspect the existing codebase carefully, especially:

- game.py
- board.py
- ai.py
- main.py

Do not rewrite the project or change its overall architecture.

Trace how coordinates are represented and converted at each stage:

1. user input
2. ship placement
3. player shots
4. AI target selection
5. AI shots
6. hit detection

Identify the coordinate inconsistency responsible for the original broken behavior I reproduced before making changes.

Then fix Task 1 with the smallest reasonable changes while preserving the existing modular structure.

Task 1 must satisfy:

- every real ship coordinate is treated as a HIT
- every known empty coordinate is treated as a MISS
- player and AI coordinates use one consistent representation internally

After making the changes:

1. run the program
2. test a known ship coordinate
3. test a known empty coordinate
4. test an invalid coordinate
5. test quitting

Do not implement Tasks 2, 3, or 4 yet.

Before finishing, briefly explain:

- what the coordinate mismatch was
- which files you changed
- why your fix resolves it
- what tests you performed

GitHub Copilot: I'll trace the coordinate path in the four named modules and check the README's Task 1 wording, then I'll identify the specific mismatch before editing. I'll also load the Python debugging guidance and use a small, reproducible run for the requested behavior checks.

Read []()

Read []()

Read []()

Read []()

Read []()

Read skill [python-fact-grounded-coding]()

Read memory

The key path is already visible: player input becomes a zero-based `(row, col)` tuple, while setup stores ship cells as one-based-looking tuples; the AI also turns its zero-based choice into a one-based string that the game later treats as an internal tuple. I'm checking those mismatches against live behavior now; the smallest likely correction is to keep all stored/returned coordinates zero-based and convert only at the user-facing input/output boundary.

Ran `pylanceRunCodeSnippet`

Completed with input: {

```
"workspaceRoot": "c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4",
"workingDirectory":
"c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4\\19_battleship",
"codeSnippet": "from game import Battleship\n\ngame = Battleship()\nuser_ship_input = (2 - 1,
2 - 1)\nprint({'setup_enemy_ships': sorted(game.enemy.ships), 'user_input_2_2_internal':
user_ship_input, 'reported_hit_at_2_2':
game.enemy.fire(user_ship_input)})\nprint({'setup_enemy_ships_include_2_2': (2, 2) in
game.enemy.ships, 'actual_intended_ship_input_3_3_internal': (3 - 1, 3 - 1),
'reported_hit_at_3_3': (3 - 1, 3 - 1) in game.enemy.ships})\n"
}
```

The runtime repro confirms the root defect: the enemy ship stored at `(2, 2)` is missed by user input `2,2` (converted to `(1, 1)`), while input `3,3` incorrectly hits it. I'm making the setup coordinates zero-based and having the AI return its already-zero-based tuple, leaving the 1-based conversion at the user-facing input/output boundary.

Apply Patch

Generating patch (32 lines) in `[]()`, `[]()`

...

...

...

...

Ran `pylanceRunCodeSnippet`

Completed with input: {

```
"workspaceRoot": "c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4",
"workingDirectory":
"c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4\\19_battleship",
"codeSnippet": "from contextlib import redirect_stdout\nfrom io import StringIO\nfrom
unittest.mock import patch\n\nfrom ai import AI\nfrom game import Battleship\n\n# Every board
cell's result must match the stored ship state.\n\ngame = Battleship()\nfor board in (game.player,
game.enemy):\n    for row in range(6):\n        for col in range(6):\n            assert board.fire((row,
col)) is ((row, col) in board.ships)\n\n# AI selections remain internal zero-based tuples and use
the same coordinates as ship state.\nai = AI()\nai.tried = {(row, col) for row in range(6) for col in
range(6) if (row, col) != (0, 0)}\nai_pos = ai.choose()\nassert ai_pos == (0, 0) and ai_pos in
```

```

game.player.ships\n\n# Run the interactive program through hit, miss, invalid input, and
quit.\noutput = StringIO()\nwith patch(\"builtins.input\", side_effect=[\"2,2\", \"6,6\", \"7,1\", \"q\"]),
redirect_stdout(output):\n    Battleship().run()\ntranscript = output.getvalue()\nassert \"HIT!\" in
transcript\nassert \"MISS!\" in transcript\nassert \"Outside board.\" in transcript\nassert
transcript.count(\"Battleship\") == 1\nprint(\"PASS: all 36 cells on each board match hit/miss
membership\")\nprint(\"PASS: AI selects internal zero-based coordinate (0, 0), recognized as a
player ship\")\nprint(\"PASS: program run handled known hit (2,2), known miss (6,6), invalid
(7,1), and quit\")\nprint(transcript)\n"
}

```

The focused run passes: all 36 cells on both boards agree with ship membership, AI targeting now yields the same tuple representation as player ships, and the game loop reports the tested hit, miss, invalid input, and clean quit. I'm doing a final error/diff check to ensure only Task 1's two coordinate boundaries changed.

Ran terminal command: `git diff -- 19_battleship/game.py 19_battleship/ai.py`

Checked `[]()`, `[]()`, no problems found

Ran terminal command: `git -C "c:\Users\Prakruthi\Desktop\SE_Experiential_Learning\Lab4" diff -- 19_battleship/game.py 19_battleship/ai.py`

****Task 1 is fixed**** in ``game.py`` and ``ai.py``.

The mismatch was that ship placement stored one-based-looking coordinates, while player input was converted to zero-based tuples. The AI also converted its zero-based selection into a one-based string, which the game parsed and checked as though it were already an internal coordinate. Ship cells and AI selections now use zero-based tuples throughout; conversion happens only when accepting or displaying user-facing coordinates. This makes hits and misses agree with the board's stored ship cells.

I ran the game loop with inputs for a known ship (``2,2``), a known empty cell (``6,6``), an invalid coordinate (``7,1``), and quit (``q``). I also checked every cell on both boards against ship membership, verified an AI-selected coordinate matches the player's internal coordinate representation, and confirmed there are no Pylance errors in the changed files. Tasks 2–4 remain untouched.

Made changes.

Task2:

User: Task 1 is now implemented and tested successfully.

Now implement Task 2 from README.md: Complete fleet and win logic.

First inspect the current implementation after the Task 1 changes and understand how ships, damage, shots, and game state are currently represented.

Implement Task 2 without undoing or weakening the Task 1 coordinate fix.

Requirements:

- support multiple ships as separate ships
- track damage separately for each individual ship
- identify when an individual ship is sunk
- prevent repeated shots at the same coordinate
- end the game when the entire enemy fleet is sunk
- provide clear HIT, MISS, and SUNK feedback

Keep the existing terminal interface and modular structure.

Do not rewrite the project from scratch.

Do not add unrelated features.

Do not add persistence, databases, CSV, JSON, SQLite, or unnecessary external dependencies.

Keep the game state in memory.

After implementation, run focused tests for:

1. hitting a ship
2. missing a ship
3. shooting the same coordinate twice
4. damaging a ship across multiple shots
5. sinking an individual ship
6. sinking the complete enemy fleet
7. invalid coordinates
8. quitting

Also verify that the Task 1 coordinate behavior still works after your changes.

At the end, briefly explain:

- how multiple ships are represented
- how damage is tracked for each ship
- how repeated shots are detected
- how an individual ship is identified as sunk

- how the complete-fleet win condition works
- which files were changed
- what tests were performed

Do not implement Tasks 3 or 4 yet.

GitHub Copilot: I'll inspect the post-Task-1 board/game code and the README's exact Task 2 criteria first, then trace the current shot state to the smallest design that can track per-ship damage without changing the coordinate convention. After that I'll implement Task 2 and exercise the required cases plus a Task 1 coordinate regression check.

Read []()

Read []()

Read []()

Read []()

The current `Board` flattens every ship into one cell set, so it can tell whether a cell is occupied but cannot identify which ship took damage or whether that ship sank. There's also a repeat-shot check against `self.player.shots`, although player shots are stored on `self.enemy`; I'll move repeat detection into `Board.fire`, store each ship's cells and hits separately, and keep the established zero-based tuple coordinates. The direct check afterward will fire each test coordinate twice/through a ship's full cell set and verify its individual and fleet outcomes.

Generating patch (80 lines) in [], []()

...

...

...

...

Ran `pylanceRunCodeSnippet`

Completed with input: {

"workspaceRoot": "c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4",

"workingDirectory":

"c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4\\19_battleship",

```

"codeSnippet": "from contextlib import redirect_stdout\nfrom io import StringIO\nfrom\nunittest.mock import patch\n\nfrom board import Board\nfrom game import Battleship\n\n# Individual hit, miss, repeated shot, multi-hit damage, and individual/full-fleet sinking.\nboard =\nBoard()\nboard.place_ship({(1, 1), (1, 2)})\nboard.place_ship({(3, 3)})\nassert board.fire((1, 1))\n== \"HIT\"\nassert board.fire((1, 1)) == \"REPEAT\"\nassert board.fire((0, 0)) == \"MISS\"\nassert\nboard.ships[0].hits == {(1, 1)} and not board.ships[1].hits\nassert board.fire((1, 2)) ==\n\"SUNK\"\nassert board.ships[0].sunk and not board.ships[1].sunk\nassert not\nboard.all_sunk()\nassert board.fire((3, 3)) == \"SUNK\"\nassert board.all_sunk()\n\n# Task 1\nregression: all configured ship cells hit; known empty cells miss.\n\ngame =\nBattleship()\nexpected_ships = {(1, 1), (1, 2), (1, 3), (3, 0), (4, 0)}\nfor row in\nrange(Board.SIZE):\n    for col in range(Board.SIZE):\n        raw_row, raw_col = row + 1, col +\n1\n        pos = (raw_row - 1, raw_col - 1)\n        result = game.enemy.fire(pos)\n        if pos in\nexpected_ships:\n            assert result in (\"HIT\", \"SUNK\"), (pos, result)\n        else:\nassert result == \"MISS\", (pos, result)\nassert game.enemy.all_sunk()\n\n# Interactive run\nexercises HIT, repeat prevention, MISS, ship SUNK, fleet win.\n\noutput = StringIO()\ninputs =\n[\"2,2\", \"2,2\", \"6,6\", \"2,3\", \"2,4\", \"5,5\", \"5,6\"]\nwith patch(\"builtins.input\", side_effect=inputs), redirect_stdout(output):\n    Battleship().run()\ntranscript =\noutput.getvalue()\nassert \"HIT!\" in transcript\nassert \"Already fired there.\" in transcript\nassert\n\"MISS!\" in transcript\nassert \"HIT! SUNK!\" in transcript\nassert \"You sank the fleet.\" in\ntranscript\n\n# Invalid coordinate then quit exits without trying another shot.\n\noutput =\nStringIO()\nwith patch(\"builtins.input\", side_effect=[\"7,1\", \"q\"]), redirect_stdout(output):\n    Battleship().run()\nassert \"Outside board.\" in output.getvalue()\n\nprint(\"PASS: hit, miss,\nrepeated shot, per-ship damage, individual sink, complete-fleet sink\")\nprint(\"PASS: Task 1\ncoordinate mapping across all cells (all ships hit; all empty cells miss)\")\nprint(\"PASS:\ninteractive game feedback, invalid coordinate, and quitting\")\nprint(transcript)\n"}

```

The first check caught an error in my regression test's expected fleet: I included the player's second ship coordinates when checking the enemy board. The implementation's board setup is separate as intended; I'm correcting that expected set and rerunning the same focused checks before inspecting anything else.

Ran `pylanceRunCodeSnippet`

Completed with input: {

"workspaceRoot": "c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4",

"workingDirectory":

"c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4\\19_battleship",

```

"codeSnippet": "from contextlib import redirect_stdout\nfrom io import StringIO\nfrom\nunittest.mock import patch\n\nfrom board import Board\nfrom game import Battleship\n\nboard\n= Board()\nboard.place_ship({(1, 1), (1, 2)})\nboard.place_ship({(3, 3)})\nassert board.fire((1, 1))\n== \"HIT\"\nassert board.fire((1, 1)) == \"REPEAT\"\nassert board.fire((0, 0)) == \"MISS\"\nassert\nboard.ships[0].hits == {(1, 1)} and not board.ships[1].hits\nassert board.fire((1, 2)) ==\n\"SUNK\"\nassert board.ships[0].sunk and not board.ships[1].sunk\nassert not\nboard.all_sunk()\nassert board.fire((3, 3)) == \"SUNK\"\nassert board.all_sunk()\n\n# Task 1

```

```

regression: every configured enemy ship cell hits and every other cell misses.\n
game = Battleship()\n
expected_enemy_ships = {(1, 1), (1, 2), (1, 3), (4, 4), (4, 5)}\n
for row in range(Board.SIZE):\n
    for col in range(Board.SIZE):\n
        pos = (row, col)\n
        result = game.enemy.fire(pos)\n
        if pos in expected_enemy_ships:\n
            assert result in ("HIT", "SUNK")\n
        else:\n
            assert result == "MISS"\n
        (pos, result)\n
game.enemy.all_sunk()\n
output = StringIO()\n
inputs = ["2,2", "2,2", "6,6", "2,3", "2,4", "5,5", "5,6"]\n
with patch("builtins.input", side_effect=inputs), redirect_stdout(output):\n
    Battleship().run()\n
transcript = output.getvalue()\n
assert "HIT!" in transcript\n
assert "Already fired there." in transcript\n
assert "MISS!" in transcript\n
assert "HIT! SUNK!" in transcript\n
assert "You sank the fleet." in transcript\n
output = StringIO()\n
with patch("builtins.input", side_effect=["7,1", "q"]), redirect_stdout(output):\n
    Battleship().run()\n
assert "Outside board." in output.getvalue()\n
print("PASS: hit, miss, repeat, per-ship damage, individual sink, complete-fleet sink")\n
print("PASS: Task 1 coordinates: every enemy ship cell hits, every empty cell misses")\n
print("PASS: interactive hit/miss/sunk feedback, invalid coordinate, and quit")\n
print(transcript)\n
}

```

The corrected focused run passes every requested case, including the two-ship game finishing only after both ships are sunk; all enemy ship coordinates still hit and every other board cell misses under Task 1's mapping. I'm checking diagnostics on the two changed modules as the final gate.

Checked `[]()`, `[]()`, no problems found

Task 2 is implemented in `board.py` and `game.py`. The Task 1 zero-based coordinate handling remains intact.

Each board now stores ships separately. Each ship tracks its own cells and hit coordinates; once all its cells are hit, it is identified as sunk. The board records fired coordinates and returns a repeat result for duplicate shots. The game reports hits, misses, and sunk ships, and ends when every enemy ship is sunk.

Focused tests passed for hits, misses, repeat shots, damage across multiple shots, sinking one ship, sinking the full fleet, invalid coordinates, and quitting. I also verified the Task 1 mapping across every board cell: ship cells hit, and empty cells miss. No Pylance errors were reported. Tasks 3 and 4 remain unimplemented.

Made changes.

Task3:

User: Tasks 1 and 2 are now implemented and tested successfully.

Now implement Task 3 from README.md: Improve the AI.

First inspect the current AI implementation after the Task 1 and Task 2 changes. Understand how the AI currently tracks tried coordinates and how the result of an actual shot is communicated back to it.

Implement Task 3 without undoing or weakening Tasks 1 or 2.

Requirements:

- after the AI gets a HIT, it should preferentially consider nearby untried cells
- the AI must never shoot the same coordinate twice
- the AI must safely handle a board with no remaining valid choices

Keep the existing terminal interface and modular structure.

Do not rewrite the project from scratch.

Do not add unrelated features.

Do not add persistence or unnecessary external dependencies.

The AI should only update its targeting state based on the result of an actual shot. Do not print or report HIT/MISS/SUNK messages while merely evaluating candidate cells.

After implementation, run focused tests for:

1. AI selects a valid untried coordinate
2. AI never selects the same coordinate twice
3. after an actual AI HIT, nearby untried cells are preferentially considered
4. already-tried nearby cells are excluded
5. the AI safely handles the case where no valid choices remain
6. existing Task 1 coordinate handling still works
7. existing Task 2 fleet, repeated-shot, sinking, and win behavior still works

At the end, explain:

- how the AI tracks tried coordinates
- how it knows when its previous shot was a HIT
- how nearby candidates are selected after a HIT
- how repeated shots are prevented
- how the no-choices case is handled
- which files were changed
- what tests were performed

Do not implement Task 4 yet.

GitHub Copilot: I'll trace the AI selection flow and the game's actual-shot result handling first, then add hit-aware targeting only where the game can report a completed shot. That keeps candidate evaluation silent and lets the AI's tried set remain the source of duplicate prevention.

Read []()

Read []()

Read []()

Read []()

The behavior is currently split: `AI.choose()` randomly selects and reserves an untried coordinate, but only `Board.fire()` knows whether that actual shot was a HIT, MISS, or SUNK, and the game never reports that result back to the AI. I'll add an AI result-report method called immediately after the real AI shot; `choose()` will prioritize untried orthogonal neighbors of its latest unsunk HIT, fall back to other untried cells, and return `None` when exhausted, which the game will handle without crashing.

Generating patch (45 lines) in [], []()

...

...

...

...

Ran `pylanceRunCodeSnippet`

Completed with input: {

 "workspaceRoot": "c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4",

 "workingDirectory":

"c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4\\19_battleship",

 "codeSnippet": "from contextlib import redirect_stdout\nfrom io import StringIO\nfrom unittest.mock import patch\n\nfrom ai import AI\nfrom board import Board\nfrom game import Battleship\n\n# AI returns valid unique coordinates, then handles exhaustion.\nai = AI(size=3)\nselected = []\nwith patch(\"ai.random.choice\", side_effect=lambda choices: choices[0]):\n while (pos := ai.choose()) is not None:\n assert 0 <= pos[0] < 3 and 0 <= pos[1] < 3\n assert pos not in selected\n selected.append(pos)\n assert ai.choose() is None\n assert len(selected) == 9\n\n# Report the result of a real HIT; next target must be an

```

untried orthogonal neighbor.\nai = AI(size=5)\nplayer_board =
Board()\nplayer_board.place_ship({(2, 2), (2, 3)})\nwith patch("ai.random.choice",
side_effect=lambda choices: (2, 2) if (2, 2) in choices else choices[0]):\n    hit_pos =
ai.choose()\n    hit_result = player_board.fire(hit_pos)\n    assert hit_result == "HIT"\n
ai.record_result(hit_pos, hit_result)\n    ai.tried.add((1, 2))\n    next_pos = ai.choose()\nassert
next_pos in {(3, 2), (2, 1), (2, 3)}\nassert next_pos not in ai.tried - {next_pos}\n\n# Results only
focus targeting after actual shots; sinking clears the target focus.\nsunk_result =
player_board.fire((2, 3))\nassert sunk_result == "SUNK"\nai.record_result((2, 3),
sunk_result)\nassert ai.last_hit is None\n\n# Task 1 regression across the full board, using
user-facing 1-based coordinates.\ngame = Battleship()\nship_cells = {cell for ship in
game.enemy.ships for cell in ship.cells}\nfor row in range(Board.SIZE):\n    for col in
range(Board.SIZE):\n        pos = (row + 1 - 1, col + 1 - 1)\n        result = game.enemy.fire(pos)\n
assert (result in ("HIT", "SUNK")) == (pos in ship_cells), (pos, result)\n\n# Task 2 board
behaviors: separate damage, repeat prevention, individual and fleet sinking.\nboard =
Board()\nboard.place_ship({(0, 0), (0, 1)})\nboard.place_ship({(1, 1)})\nassert board.fire((0, 0))
== "HIT"\nassert board.fire((0, 0)) == "REPEAT"\nassert board.fire((0, 1)) ==
"SUNK"\nassert board.ships[0].sunk and not board.ships[1].sunk and not
board.all_sunk()\nassert board.fire((1, 1)) == "SUNK" and board.all_sunk()\n\n# Full game fleet
win and repeat/invalid/quit paths.\noutput = StringIO()\nwith patch("builtins.input",
side_effect=["2,2", "2,3", "2,4", "5,5", "5,6"]), redirect_stdout(output):\n
Battleship().run()\nassert "You sank the fleet." in output.getvalue()\noutput = StringIO()\nwith
patch("builtins.input", side_effect=["6,6", "6,6", "7,1", "q"]), redirect_stdout(output):\n
Battleship().run()\nassert "Already fired there." in output.getvalue()\nassert "Outside board." in
output.getvalue()\n\n# The game safely handles an exhausted AI without firing an invalid
coordinate.\noutput = StringIO()\ngame = Battleship()\ngame.ai.tried = {(row, col) for row in
range(Board.SIZE) for col in range(Board.SIZE)}\nwith patch("builtins.input",
side_effect=["6,6", "q"]), redirect_stdout(output):\n    game.run()\nassert "AI has no valid
shots remaining." in output.getvalue()\n\nprint("PASS: valid unique AI picks and no-choices
exhaustion")\nprint("PASS: actual HIT biases toward untried neighbors; tried neighbors
excluded; SUNK clears focus")\nprint("PASS: Task 1 coordinates and Task 2
damage/repeat/sinking/fleet-win regressions")\nprint("PASS: invalid input, quitting, and
exhausted-AI game path")\n"
}

```

The first focused suite passes. One detail deserves tightening against your explicit “actual shot” constraint: `choose()` still marked a coordinate tried before `Board.fire()` returned its result. I’m moving that update into `record\_result()`, so candidate selection only proposes a coordinate and tried/hit targeting state changes only after a real board result; the game already reports the result immediately.

Generating patch (15 lines) in `[]()`

...


```

shot results\\")\nprint("PASS: actual HIT prioritizes untried neighbors; already-tried neighbors
are excluded\\")\nprint("PASS: no-choice handling and Task 1/2 regression
behaviors\\")\nprint("PASS: fleet win, repeated shot, invalid coordinate, and quitting\\")\n"
}

```

The revised suite passes with the stricter state timing: selected coordinates enter `tried` only after `Board.fire()` returns an actual HIT, MISS, or SUNK result. Neighbor targeting excludes a previously fired adjacent cell, and the Task 1/2 regression checks still pass; I'm running the final diagnostics now.

Checked `[]()`, `[]()`, no problems found

Task 3 is implemented in `ai.py` and `game.py`. Tasks 1 and 2 remain intact.

The AI tracks completed shot coordinates in `tried`. After `Board.fire()` returns an actual result, the game passes it to `record\_result()`. A HIT becomes the AI's targeting focus; it prefers random, in-bounds orthogonal neighbors that are not in `tried`. A SUNK clears that focus. If no adjacent options remain, it chooses from other untried cells. When none remain, `choose()` returns `None`, and the game safely skips the AI's turn.

Focused tests passed for valid unique selections, hit-based neighboring priority, exclusion of tried neighbors, and the no-choices case. I also verified Task 1 coordinate behavior; Task 2 damage tracking, repeat prevention, sinking, and fleet win; plus invalid input and quitting. Diagnostics reported no errors. Task 4 was not implemented.

Made changes.

Task4:

User: Tasks 1, 2, and 3 are now implemented and tested successfully.

Now implement Task 4 from README.md: Shot-level feedback.

First inspect the current implementation after Tasks 1–3 and trace exactly where:

- the player fires a shot
- the AI selects a candidate
- the actual shot is executed
- Board.fire() determines the result
- the result is displayed to the user
- the AI receives the result

Implement Task 4 without undoing or weakening Tasks 1, 2, or 3.

Requirements:

- Every actual player shot must produce exactly one appropriate result message: HIT, MISS, or SUNK.
- Every actual AI shot must produce exactly one appropriate result message: HIT, MISS, or SUNK.
- Internal AI candidate selection or evaluation must NOT produce HIT/MISS/SUNK messages.
- Repeated shots must not be treated as new shots or produce duplicate shot-result feedback.
- A SUNK message should only appear when an individual ship is actually sunk.
- Fleet completion should still produce the existing clear win message.
- Keep the existing terminal interface and modular structure.

Do not rewrite the project from scratch.

Do not add unrelated features.

Do not add persistence, databases, CSV, JSON, SQLite, or unnecessary dependencies.

Do not change the coordinate representation established in Task 1.

Do not weaken the fleet, repeated-shot, sinking, or AI behavior implemented in Tasks 2 and 3.

Pay special attention to the distinction between:

1. choosing/evaluating a candidate coordinate, and
2. actually firing at that coordinate.

Only the second should generate shot feedback.

After implementation, run focused tests for:

1. A real player HIT produces exactly one HIT message.
2. A real player MISS produces exactly one MISS message.
3. A real player shot that sinks a ship produces exactly one SUNK message.
4. A real AI HIT produces exactly one HIT message.
5. A real AI MISS produces exactly one MISS message.
6. A real AI shot that sinks a ship produces exactly one SUNK message.
7. AI candidate selection itself produces no HIT/MISS/SUNK output.
8. Repeated shots do not generate new shot-result feedback.
9. The complete enemy fleet still ends the game correctly.
10. Task 1 coordinate handling still works.
11. Task 2 fleet, damage, repeated-shot, sinking, and win behavior still works.
12. Task 3 AI hit-based targeting, tried-coordinate tracking, and no-valid-choice handling still works.
13. Invalid coordinates and quitting still work.

For feedback-count tests, verify the number of actual shot-result messages rather than only checking that a word appears somewhere in the output.

At the end, briefly explain:

- where shot feedback is generated

- how you ensure exactly one feedback message per actual shot
- how candidate evaluation remains silent
- how repeated shots are handled
- how SUNK feedback is distinguished from HIT
- which files were changed
- what tests were performed

Do not add any features beyond Task 4.

GitHub Copilot: I'll trace the current result path from both callers through `Board.fire()` and compare it with Task 4's required message semantics. Then I'll make the smallest change that centralizes one user-facing result per executed shot while leaving AI selection silent and preserving the existing repeat, sunk, fleet-win, and coordinate behavior.

Read `[]()`

Read `[]()`

Read `[]()`, lines 35 to 94

Read

The current player path prints `'HIT! SUNK!'` for a sunk ship, while the AI path prints generic phrases and omits misses, so neither side has exactly one standardized result category per real shot. I'll add one shared formatter called only after `Board.fire()` returns a non-repeat result; AI reporting will remain after the actual fire and before/alongside its state update, while candidate selection stays unchanged and silent.

Generating patch (37 lines) in

...

...

Ran ``pylanceRunCodeSnippet``

Completed with input: {

 "workspaceRoot": "c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4",

 "workingDirectory":

"c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4\\19_battleship",

 "codeSnippet": "from contextlib import redirect_stdout\nfrom io import StringIO\nfrom unittest.mock import patch\n\nfrom ai import AI\nfrom board import Board\nfrom game import Battleship\n\n\ndef run_game(inputs, setup=None, ai_choice=None):\n output = StringIO()\n"

```

with patch("builtins.input", side_effect=inputs), redirect_stdout(output):\n    if setup is None:\n
game = Battleship()\n    else:\n        with patch.object(Battleship, "_setup", setup):\n
game = Battleship()\n        choice_patch = patch.object(AI, "choose", return_value=ai_choice) if
ai_choice is not None else patch.object(AI, "choose", return_value=None)\n        with
choice_patch:\n            game.run()\n    return output.getvalue()\n\n\ndef feedback(output):\n
return [line for line in output.splitlines() if line.startswith(("You: ", "AI: "))]\n\n# 1-3: player HIT,
MISS, and individual ship SUNK each emit exactly one category.\nassert
feedback(run_game(["2,2", "q"])) == ["You: HIT!"]\nassert feedback(run_game(["6,6", "q"]))
== ["You: MISS!"]\nplayer_sunk_output = run_game(["2,2", "2,3", "2,4", "q"])\nassert
feedback(player_sunk_output) == ["You: HIT!", "You: HIT!", "You: SUNK!"]\nassert
player_sunk_output.count("SUNK!") == 1\n\n# 4-6: AI HIT, MISS, and SUNK each emit exactly
one category after an actual Board.fire.\ndef setup_ai_two_cell_ship(game):\n
game.player.place_ship([(0, 0), (0, 1)])\n    game.enemy.place_ship([(1, 1), (1, 2)])\n\ndef
setup_ai_one_cell_ship(game):\n    game.player.place_ship([(0, 0)])\n
game.enemy.place_ship([(1, 1), (1, 2)])\n\nassert feedback(run_game(["6,6", "q"],
setup_ai_two_cell_ship, (0, 0))) == ["You: MISS!", "AI: HIT!"]\nassert
feedback(run_game(["6,6", "q"], setup_ai_two_cell_ship, (5, 5))) == ["You: MISS!", "AI:
MISS!"]\nassert feedback(run_game(["6,6", "q"], setup_ai_one_cell_ship, (0, 0))) == ["You:
MISS!", "AI: SUNK!"]\n\n# 7: candidate selection itself must be silent.\noutput =
StringIO()\nwith redirect_stdout(output):\n    selected = AI().choose()\nassert selected is not
None and output.getvalue() == ""\n\n# 8: repeated player shot is not a new result-bearing
shot.\nrepeat_output = run_game(["6,6", "6,6", "q"])\nassert feedback(repeat_output) ==
["You: MISS!"]\nassert repeat_output.count("Already fired there.") == 1\n\n# 9-11: full-fleet
win, Task 1 coordinates, and Task 2 shot/ship state.\nfleet_output = run_game(["2,2", "2,3",
"2,4", "5,5", "5,6"])\nassert feedback(fleet_output) == ["You: HIT!", "You: HIT!", "You:
SUNK!", "You: HIT!", "You: SUNK!"]\nassert fleet_output.count("You sank the fleet.") ==
1\n\nboard = Board()\nboard.place_ship([(1, 1), (1, 2)])\nboard.place_ship([(4, 4)])\nassert
board.fire((1, 1)) == "HIT"\nassert board.fire((1, 1)) == "REPEAT"\nassert board.fire((1, 2)) ==
"SUNK"\nassert board.ships[0].sunk and not board.ships[1].sunk and not
board.all_sunk()\nassert board.fire((4, 4)) == "SUNK" and board.all_sunk()\n\nngame =
Battleship()\nship_cells = {cell for ship in game.enemy.ships for cell in ship.cells}\nfor row in
range(Board.SIZE):\n    for col in range(Board.SIZE):\n        pos = (row, col)\n        result =
game.enemy.fire(pos)\n        assert (result in ("HIT", "SUNK")) == (pos in ship_cells), (pos,
result)\n\n# 12: AI result tracking, adjacent preference, tried exclusion, and exhaustion.\nai =
AI(size=3)\nassert ai.choose() is not None\nai = AI(size=3)\nai.record_result((1, 1),
"HIT")\nai.record_result((0, 1), "MISS")\nwith patch("ai.random.choice", side_effect=lambda
candidates: candidates[0]):\n    adjacent = ai.choose()\nassert adjacent in {(2, 1), (1, 0), (1,
2)}\nai = AI(size=2)\nfor row in range(2):\n    for col in range(2):\n        pos = (row, col)\n
assert ai.choose() == pos\n        ai.record_result(pos, "MISS")\nassert ai.choose() is None\n\n#
13: invalid input and quitting produce no shot-result feedback.\ninvalid_quit_output =
run_game(["7,1", "q"])\nassert "Outside board." in invalid_quit_output\nassert
feedback(invalid_quit_output) == []\nassert "Battleship" in invalid_quit_output\n\nprint("PASS:
exact one-message counts for player/AI HIT, MISS, SUNK, and repeated shots")\nprint("PASS:
silent candidate choice; fleet win; Task 1/2/3 regressions; invalid input and quit")\n"

```

```
}
```

The feedback-count cases themselves passed, including player/AI HIT, MISS, SUNK, repeat suppression, and fleet completion. The combined run stopped at a Task 3 regression assertion that assumed random AI choices arrive in row order; I'm making that check deterministic and rerunning the full suite.

Ran `pylanceRunCodeSnippet`

Completed with input: {

```
"workspaceRoot": "c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4",
"workingDirectory":
"c:\\Users\\Prakruthi\\Desktop\\SE_Experiential_Learning\\Lab4\\19_battleship",
"codeSnippet": "from contextlib import redirect_stdout\nfrom io import StringIO\nfrom
unittest.mock import patch\n\nfrom ai import AI\nfrom board import Board\nfrom game import
Battleship\n\n\n\ndef run_game(inputs, setup=None, ai_choice=None):\n    output = StringIO()\n    with patch(\"builtins.input\", side_effect=inputs), redirect_stdout(output):\n        if setup is None:\n            game = Battleship()\n        else:\n            with patch.object(Battleship, \"_setup\", setup):\n                game = Battleship()\n        choice_patch = patch.object(AI, \"choose\", return_value=ai_choice) if
ai_choice is not None else patch.object(AI, \"choose\", return_value=None)\n        with
choice_patch:\n            game.run()\n    return output.getvalue()\n\n\ndef feedback(output):\n    return [line for line in output.splitlines() if line.startswith((\"You: \", \"AI: \"))]\n\n\nassert
feedback(run_game([\"2,2\", \"q\"])) == [\"You: HIT!\"]\nassert feedback(run_game([\"6,6\", \"q\"]))
== [\"You: MISS!\"]\nplayer_sunk_output = run_game([\"2,2\", \"2,3\", \"2,4\", \"q\"])\nassert
feedback(player_sunk_output) == [\"You: HIT!\", \"You: HIT!\", \"You: SUNK!\"]\nassert
player_sunk_output.count(\"SUNK!\") == 1\n\ndef setup_ai_two_cell_ship(game):\n    game.player.place_ship(({0, 0}, {0, 1}))\n    game.enemy.place_ship(({1, 1}, {1, 2}))\n\ndef
setup_ai_one_cell_ship(game):\n    game.player.place_ship(({0, 0}))\n    game.enemy.place_ship(({1, 1}, {1, 2}))\n\n\nassert feedback(run_game([\"6,6\", \"q\"],
setup_ai_two_cell_ship, (0, 0))) == [\"You: MISS!\", \"AI: HIT!\"]\nassert
feedback(run_game([\"6,6\", \"q\"], setup_ai_two_cell_ship, (5, 5))) == [\"You: MISS!\", \"AI:
MISS!\"]\nassert feedback(run_game([\"6,6\", \"q\"], setup_ai_one_cell_ship, (0, 0))) == [\"You:
MISS!\", \"AI: SUNK!\"]\n\n\noutput = StringIO()\nwith redirect_stdout(output):\n    selected =
AI().choose()\nassert selected is not None and output.getvalue() == \"\"\n\n\nrepeat_output =
run_game([\"6,6\", \"6,6\", \"q\"])\nassert feedback(repeat_output) == [\"You: MISS!\"]\nassert
repeat_output.count(\"Already fired there.\") == 1\n\n\nfleet_output = run_game([\"2,2\", \"2,3\",
\"2,4\", \"5,5\", \"5,6\"])\nassert feedback(fleet_output) == [\"You: HIT!\", \"You: HIT!\", \"You:
SUNK!\", \"You: HIT!\", \"You: SUNK!\"]\nassert fleet_output.count(\"You sank the fleet.\") ==
1\n\n\nboard = Board()\nboard.place_ship(({1, 1}, {1, 2}))\nboard.place_ship(({4, 4}))\nassert
board.fire((1, 1)) == \"HIT\"\nassert board.fire((1, 1)) == \"REPEAT\"\nassert board.fire((1, 2)) ==
\"SUNK\"\nassert board.ships[0].sunk and not board.ships[1].sunk and not
board.all_sunk()\nassert board.fire((4, 4)) == \"SUNK\" and board.all_sunk()\n\n\n\ngame =
Battleship()\nship_cells = {cell for ship in game.enemy.ships for cell in ship.cells}\n\nfor row in
range(Board.SIZE):\n    for col in range(Board.SIZE):\n        pos = (row, col)\n        result =
game.enemy.fire(pos)\n        assert (result in (\"HIT\", \"SUNK\")) == (pos in ship_cells), (pos,
```

```

result)\n\nai = AI(size=3)\nwith patch(\"ai.random.choice\", side_effect=lambda candidates:
candidates[0]):\n    first = ai.choose()\n    assert first is not None\n    player_board =
Board()\n    player_board.place_ship({(1, 1), (1, 2)})\n    ai = AI(size=3)\n    with
patch(\"ai.random.choice\", side_effect=lambda candidates: (1, 1) if (1, 1) in candidates else
candidates[0]):\n        hit = ai.choose()\n        hit_result = player_board.fire(hit)\n        assert hit_result ==
\"HIT\"\n        ai.record_result(hit, hit_result)\n        tried_neighbor = (0, 1)\n
ai.record_result(tried_neighbor, player_board.fire(tried_neighbor))\n        nearby =
ai.choose()\n        assert nearby in {(2, 1), (1, 0), (1, 2)}\n\n    ai = AI(size=2)\n    selected = set()\n    with
patch(\"ai.random.choice\", side_effect=lambda candidates: candidates[0]):\n        while (pos :=
ai.choose()) is not None:\n            assert pos not in selected\n            selected.add(pos)\n
ai.record_result(pos, \"MISS\")\n        assert len(selected) == 4 and ai.choose() is
None\n\n    invalid_quit_output = run_game([\"7,1\", \"q\"])\n    assert \"Outside board.\" in
invalid_quit_output\n    assert feedback(invalid_quit_output) == []\n\n    print(\"PASS: exact
one-message counts for player/AI HIT, MISS, SUNK, and repeated shots\")\n    print(\"PASS: silent
candidate selection; fleet win; Task 1/2/3 regressions\")\n    print(\"PASS: invalid coordinates and
quitting\")\n\n}

```

The deterministic rerun passes all 13 requested checks. Feedback is counted by exact
`You:`/`AI:` result lines: each actual shot has one category, repeated shots add none, and fleet
completion still emits its separate win message. I'm checking diagnostics on the changed game
module before wrapping up.

Checked `[]()`, no problems found

Task 4 is implemented in `game.py`.

Player and AI shot results now use one shared formatter, called only after `Board.fire()` returns
an actual result. AI candidate selection remains silent. Repeated shots are handled before the
formatter, so they produce no new HIT/MISS/SUNK feedback. `SUNK` is reported only when
`Board.fire()` identifies an individual ship as sunk; the existing fleet-win message remains
separate.

All 13 requested test areas passed. Feedback-count checks verified exactly one result line per
actual player or AI HIT, MISS, and SUNK; repeated shots produced none. I also verified silent
candidate selection, fleet completion, Task 1 coordinate behavior, Task 2 fleet and damage
behavior, Task 3 targeting and exhaustion behavior, invalid coordinates, and quitting.
Diagnostics reported no errors.

Made changes.